

DevOps Implementation Plan

Python + AI/Data Project | GitHub + GitHub Actions + Kubernetes + PostgreSQL

Purpose

This document is a practical DevOps plan for a project with approximately 4–5 Python repositories, PostgreSQL, AI/data-processing workloads, GitHub for source control, GitHub Actions for CI/CD, an AWS account, Kubernetes as the deployment platform, and SonarQube for code-quality checks. The goal is to build a repeatable delivery process rather than creating separate, inconsistent pipelines for each repository.

How we will approach it

First understand what each repository does, then standardize the development process, establish the AWS/Kubernetes foundation, build one good CI/CD pipeline, validate it with one repository, and finally apply the same pattern to the remaining repositories.

1. Initial Discovery

Before changing the application, collect the following information for each repository:

- Repository purpose and ownership
- Python version and dependency-management approach
- How the application starts and whether it is an API, worker, scheduled job, or data/AI process
- PostgreSQL connection and migration requirements
- External APIs, model providers, storage, queues, or other dependencies
- Expected CPU, memory, storage, processing time, and whether GPU is required
- Current testing and deployment process
- Configuration and secrets currently used
- Development, staging, and production requirements

2. Repository Standardization

Create a common baseline across all 4–5 Python repositories without unnecessarily rewriting application code.

- Use a consistent branch and pull-request process.
- Protect the main branch and require successful CI before merge.
- Add or standardize tests using pytest.
- Use Ruff for Python linting/formatting where suitable.
- Add dependency/security checks such as pip-audit and Bandit.
- Add a Dockerfile for applications that will run as containers.
- Keep README and runbook information in each repository.
- Keep CI/CD configuration under .github/workflows.

3. Code Quality with SonarQube

SonarQube will be part of the pull-request quality process. The pipeline should run the SonarQube analysis and then evaluate the Quality Gate. A failed quality gate should prevent the change from moving forward.

Quality checks should cover:

- Code smells and maintainability
- Bugs and reliability issues
- Security issues
- Duplicated code
- Test/coverage reporting where configured
- Quality Gate status

SonarSource provides a GitHub Actions quality-gate action that is used after the SonarQube analysis step. The analysis must complete before the Quality Gate check is evaluated. ■cite■turn0search6■

4. CI Pipeline — Pull Request

Every pull request should follow the same basic validation flow:

1. Checkout source
2. Set up Python
3. Install dependencies
4. Run unit tests
5. Run Ruff
6. Run Bandit
7. Run pip-audit
8. Run SonarQube analysis
9. Check SonarQube Quality Gate
10. Build validation

GitHub Actions workflows are stored in `.github/workflows`. GitHub also supports reusable workflows, which is useful here because the same CI pattern can be shared across multiple repositories instead of maintaining five unrelated pipelines.

■cite■turn0search0■turn0search4■

5. Containerization

For services and workers that will run on Kubernetes, standardize Docker images. Images should be built consistently in CI, tagged using the commit SHA or release version, scanned, and then pushed to Amazon ECR.

- Use a small, appropriate Python base image.
- Run the container as a non-root user where practical.
- Use `.dockerignore`.
- Do not place secrets inside the image.
- Use immutable image tags for deployments.
- Scan images with Trivy before deployment.

6. AWS + Kubernetes Foundation

Kubernetes will be the application deployment platform. On AWS, the preferred starting point is a managed Kubernetes service such as Amazon EKS so that the team can focus on workloads while AWS manages the Kubernetes control plane.

- Create separate development, staging, and production boundaries as appropriate.

- Create the required networking, IAM, security groups, and cluster access controls.
- Use Amazon ECR as the container image registry.
- Keep PostgreSQL private and restrict access to the application workloads.
- Use AWS Secrets Manager or an equivalent approved secret-management approach.
- Use Terraform to create and manage AWS infrastructure and avoid manual configuration drift.

7. Kubernetes Deployment Standards

Each application should have Kubernetes manifests or a Helm-based deployment package.

- Deployment
- Service
- ConfigMap for non-secret configuration
- Secret integration for sensitive values
- Resource requests and limits
- Liveness, readiness, and startup probes where applicable
- Horizontal Pod Autoscaler where workload characteristics justify it
- Pod disruption and availability settings where required

Kubernetes supports rolling updates for Deployments, allowing old Pods to be replaced progressively, and provides startup, liveness, and readiness probes for application health. ■cite■turn0search2■turn0search9■

8. CD Pipeline

After CI passes and the code is merged, the deployment process should be automated.

1. Build Docker image
2. Run container/security scan
3. Push image to Amazon ECR
4. Update Kubernetes deployment with the new image
5. Wait for rollout completion
6. Run basic smoke/health validation
7. Promote to staging
8. Production deployment through an approval-controlled environment

9. Environment Strategy

Environment	Purpose	Deployment
Development	Developer/integration testing	Automatic after approved CI or controlled trigger
Staging	Production-like validation	Controlled promotion
Production	Live workload	Approval-controlled deployment

10. PostgreSQL Plan

Use a managed PostgreSQL service such as Amazon RDS for production unless there is an existing reason to manage PostgreSQL ourselves. Keep the database private, restrict network access, enable backups, and document the migration process.

- Use Alembic or the project's existing migration framework.
- Do not store database passwords in source code.
- Separate database credentials/configuration by environment.
- Define backup and restore expectations.
- Test database migrations before production.

11. Infrastructure as Code

Use Terraform for AWS and other infrastructure that should be reproducible.

- AWS networking and supporting resources
- EKS and related configuration
- ECR repositories
- IAM roles and policies
- RDS
- S3 and other required services
- Monitoring and supporting infrastructure

12. Security

- Use GitHub branch protection and required status checks.
- Use GitHub Actions OIDC for AWS access rather than long-lived AWS credentials where supported.
- Apply least-privilege IAM.
- Keep PostgreSQL private.
- Store secrets in an approved secret manager.
- Scan source/dependencies/images/IaC as part of CI where practical.
- Use dependency updates and vulnerability review as an ongoing process.

13. Monitoring and Operations

- Centralize application and Kubernetes logs.
- Monitor CPU, memory, Pod restarts, application errors, database health, and request latency.
- Use health endpoints for services where applicable.
- Start with AWS-native monitoring and introduce OpenTelemetry/Grafana where the team needs deeper observability.
- Define alerts for production-impacting conditions.
- Maintain a simple rollback/runbook for each production service.

14. Recommended Open-Source Tooling

Area	Recommended tool
Source control	GitHub
CI/CD	GitHub Actions
Code quality	SonarQube

Area	Recommended tool
Python tests	pytest
Python lint/format	Ruff
Python security	Bandit
Dependency audit	pip-audit
Container/IaC security	Trivy
Containers	Docker
Infrastructure as Code	Terraform
Kubernetes packaging	Helm
Observability	OpenTelemetry + Grafana where required

15. Implementation Order

Phase 1 — Repository discovery and application understanding

Phase 2 — GitHub standards, branch protection and repository baseline

Phase 3 — SonarQube setup and quality-gate integration

Phase 4 — Dockerization and security scanning

Phase 5 — AWS foundation and Terraform

Phase 6 — Kubernetes/EKS foundation

Phase 7 — Deploy one Python repository end-to-end

Phase 8 — Create reusable GitHub Actions CI/CD workflows

Phase 9 — Apply the standard pipeline to the remaining repositories

Phase 10 — Monitoring, alerts, backup/restore and operational runbooks

16. Definition of Done

The DevOps implementation should be considered complete when:

- All active repositories are identified and documented.
- Pull requests automatically run tests, security checks and SonarQube Quality Gate.
- Applications are reproducibly built as container images.
- Images are stored in ECR and scanned before deployment.
- Infrastructure is managed through Terraform.
- Applications are deployed to Kubernetes through GitHub Actions.
- Development, staging and production promotion is defined.
- PostgreSQL is secured, backed up and has a documented migration process.
- Secrets are not stored in source code or container images.
- Production deployments have health checks and a rollback procedure.
- Logs, metrics and alerts are available for the production workloads.

17. Team Discussion / Inputs Required

Before implementation starts, the development team should provide the repository list and basic details for each application. The biggest decisions still dependent on application behavior are the Kubernetes workload type, CPU/memory requirements, AI/GPU requirements, database usage, external integrations, and the expected production traffic.

Recommended first milestone: take one representative Python repository, make the complete path work — GitHub → CI → SonarQube → security scan → Docker → ECR → Kubernetes → smoke test — and use that as the standard for the remaining repositories.

Reference notes

GitHub documents reusable workflows for reducing duplicated CI/CD configuration across repositories. Kubernetes documents rolling updates and container health probes. SonarSource documents the GitHub Actions Quality Gate integration. These references support the approach used in this plan. ■cite■turn0search0■turn0search2■turn0search6■turn0search9■